Automação de Testes de API

Fundamentos, ferramentas do mercado e demonstração prática com Postman.

Da tela pro back-end

Por que testar API também

- A tela muda de layout toda semana — a API por trás dela costuma mudar bem menos
- Testar API é mais rápido e mais barato do que testar pela tela
- Dá pra pegar bug antes de a tela existir — o back-end fica pronto primeiro
- Muita regra de negócio vive só na API e nunca aparece direto na interface

> *É a mesma pirâmide de sempre: a base (API/integração) sustenta o topo (E2E pela tela).*

Alguém já usou uma API sem saber?

- Todo app de celular, todo site que carrega dados — tem uma API conversando por trás
- Ver a previsão do tempo, pagar com Pix, pedir comida — tudo isso é uma chamada de API

Fundamentos que vamos usar

O que é uma API

- API é a sigla de "Application Programming Interface" — Interface de Programação de Aplicações
- Um contrato: um lado pede (cliente), o outro responde (servidor)
- O app do banco não guarda seu saldo — ele pergunta pra API do banco toda vez
- A gente não testa a tela nesse caso — testa a conversa entre os dois lados
- Se a API responder errado, a tela mais bonita do mundo não salva o sistema

> *A maioria das APIs que vamos ver segue o estilo REST (Representational State Transfer) — daí o nome de ferramentas como REST Assured.*

Anatomia de uma requisição HTTP

- HTTP (HyperText Transfer Protocol) é o protocolo que o navegador e a API usam pra conversar
- Método — o verbo da ação (buscar, criar, atualizar, apagar)
- URL (Uniform Resource Locator) / endpoint — o endereço do recurso (ex.: /usuarios/1)
- Headers — metadados da chamada (autenticação, tipo de conteúdo)
- Body — os dados enviados junto (só faz sentido em alguns métodos)

> *A resposta segue essa mesma lógica: status code, headers e um body de volta.*

• VOCABULÁRIO BÁSICO

Os métodos HTTP mais usados

MÉTODO	USADO PARA	TEM CORPO?	IDEMPOTENTE?
GET	Buscar um recurso	Não	Sim
POST	Criar um recurso novo	Sim	Não
PUT	Substituir um recurso inteiro	Sim	Sim
PATCH	Atualizar parte de um recurso	Sim	Não
DELETE	Remover um recurso	Não, geralmente	Sim

Idempotente = repetir a mesma chamada várias vezes dá o mesmo resultado final.

Status codes que todo QA deveria reconhecer de cara

- 2xx — deu certo (200 OK, 201 criado, 204 sem conteúdo pra devolver)
- 4xx — erro de quem pediu (400 requisição inválida, 401 sem login, 403 sem permissão, 404 não existe)
- 5xx — erro de quem respondeu (500 o servidor quebrou no meio do processo)
- O código importa tanto quanto o corpo da resposta — os dois fazem parte do teste

JSON: o idioma das APIs

- JSON (JavaScript Object Notation) é o formato de texto que praticamente toda API usa pra trocar dados — pares de chave e valor
- Objetos { } agrupam campos de uma coisa só; listas [ ] agrupam várias
- Testar API é, na prática, comparar o JSON que veio com o JSON que você esperava

```
{
  "id": 1,
  "nome": "Ana",
  "ativo": true,
  "papeis": ["aluno", "monitor"]
}
```

Panorama do mercado

Duas famílias de ferramentas

- Interface gráfica (GUI, de Graphical User Interface) — Postman, Insomnia: clica, monta a requisição, vê a resposta na tela
- Código — REST Assured, PactumJS, Supertest, Pytest + Requests, Robot Framework e Karate: os testes podem ser executados repetidamente
- Não é uma substituindo a outra — GUI ajuda a explorar a API, código é o que garante ela continuar funcionando

---

PM

• DESDE 2012 · O MAIS USADO DO MERCADO

Postman

- Nasceu como projeto pessoal pra facilitar o dia a dia com APIs, hoje é o padrão de fato do mercado
- Interface gráfica completa: monta, salva e organiza requisições em coleções
- Newman roda as coleções do Postman via linha de comando, direto no CI/CD (Integração Contínua / Entrega Contínua)
- Oferece recursos para organizar coleções, ambientes, variáveis e scripts de validação

> *Ponto de atenção: boa parte dos recursos completos hoje pede conta e nuvem da Postman.*

IN

• DESDE 2015 · HOJE MANTIDO PELA KONG

Insomnia

- Alternativa mais leve ao Postman, com interface mais simples
- Adquirido pela Kong em 2019; código aberto sob licença Apache-2.0
- Fala REST, GraphQL, gRPC (protocolo de chamadas remotas de alta performance, do Google), WebSockets e Server-Sent Events no mesmo cliente
- Guarda coleções localmente ou sincroniza por Git — não depende de nuvem por padrão

---

RA

• DESDE 2010 · PADRÃO EM TIMES JAVA

REST Assured

- Biblioteca Java, não uma interface gráfica — o teste é código desde o início
- DSL (Domain-Specific Language, uma linguagem própria pra um contexto específico) fluente no estilo given/when/then
- Roda sobre a JVM (Java Virtual Machine) — a mesma máquina virtual de qualquer projeto Java
- Ainda é o padrão de facto pra quem já testa em Java
- Setup pesa mais: exige projeto Java configurado (Maven/Gradle) antes do primeiro teste

RF

• DESDE 2008 · KEYWORD-DRIVEN

Robot Framework

- Framework de automação baseado em palavras-chave; para APIs, pode usar a biblioteca RequestsLibrary
- Sintaxe tabular por palavras-chave, com suporte a bibliotecas Python
- Bom em squads mistos, com gente de negócio revisando os testes
- Camada a mais entre o teste e o código costuma custar flexibilidade em casos complexos

---

NW

• LINHA DE COMANDO · INTEGRAÇÃO CONTÍNUA

Postman + Newman

- Postman permite montar requisições, organizar coleções e validar respostas com scripts JavaScript
- Newman executa coleções do Postman pela linha de comando, sem precisar clicar na interface
- Variáveis e ambientes ajudam a separar URL-base, dados e configurações por ambiente
- Ponto de atenção: é importante proteger tokens e evitar depender de dados pessoais ou de ambientes instáveis

> *Uma boa opção quando a equipe quer começar pela interface gráfica e depois automatizar a execução das coleções.*

ST

• JAVASCRIPT / TYPESCRIPT · NODE.JS

Supertest

- Biblioteca para fazer requisições HTTP e testar respostas em projetos JavaScript/TypeScript
- É frequentemente usada junto com frameworks de teste como Jest ou Mocha
- Permite verificar status, cabeçalhos e conteúdo da resposta usando código
- É especialmente conveniente para equipes que trabalham com aplicações e APIs em Node.js

> *A equipe precisa configurar o projeto e o runner de testes; Supertest é uma biblioteca, não uma interface gráfica completa.*

PY

• PYTHON · ECOSSISTEMA DE TESTES

Pytest + Requests

- Requests é uma biblioteca Python para enviar requisições HTTP
- Pytest organiza e executa testes, identifica falhas e apresenta resultados
- Fixtures podem compartilhar configurações, autenticação e preparação de dados entre testes
- A combinação oferece flexibilidade, mas exige que a equipe estruture e mantenha o código

> *Boa escolha para equipes que já utilizam Python e desejam controle sobre a organização dos testes.*

K

• DSL DECLARATIVA · API E INTEGRAÇÃO

Karate

- Framework que permite escrever cenários de teste com uma sintaxe declarativa, inspirada em Gherkin
- Oferece recursos para enviar requisições, validar respostas e trabalhar com dados
- Pode reduzir a quantidade de código explícito em testes comuns de API
- A equipe precisa aprender a sintaxe e avaliar como o framework se encaixa no projeto

> *É uma opção integrada para quem busca escrever cenários de API de forma declarativa.*

• PANORAMA DO MERCADO

Lado a lado

FERRAMENTA	TIPO	LINGUAGEM	PERFIL IDEAL
Postman + Newman	Coleções + CLI	Scripts JavaScript opcionais	Exploração manual e execução automatizada
REST Assured	Biblioteca de código	Java	Projetos que usam Java/JVM
PactumJS	Biblioteca de código	JavaScript / TypeScript	Testes de API com sintaxe encadeada
Supertest	Biblioteca de código	JavaScript / TypeScript	Testar APIs Node.js, especialmente HTTP
Robot Framework + RequestsLibrary	Framework por keywords	Robot Framework / Python	Testes legíveis e organizados em tabelas
Pytest + Requests	Bibliotecas de código	Python	Flexibilidade e ecossistema Python
Karate	Framework de testes	DSL própria, sintaxe semelhante a Gherkin	API, asserções e cenários declarativos

Panorama construído a partir da documentação oficial de cada projeto — ferramentas evoluem rápido, vale sempre checar a fonte antes de decidir num projeto real.

• TENDÊNCIA ATUAL

IA mudando os testes de API

- Geração automática de teste a partir da própria especificação da API — um arquivo OpenAPI/Swagger vira caso de teste sozinho
- Manutenção automática ("self-healing"): quando o formato da resposta muda, a ferramenta sugere o ajuste no teste
- Geração automática de massa de dados de teste realista, sem montar tudo na mão
- Postman e outras ferramentas do mercado já embarcam esses recursos de IA nativamente

> *Isso acelera a escrita do teste — mas decidir o que testar e por quê continua sendo trabalho do QA.*

NOVIDADE DO ECOSSISTEMA

APIs e o Model Context Protocol (MCP)

- MCP (Model Context Protocol, ou Protocolo de Contexto de Modelo) é um padrão pra um agente de IA descobrir e usar ferramentas
- Na prática, a "ferramenta" que o agente usa costuma ser uma API REST por trás — o MCP não substitui a API, ele padroniza a conversa com ela
- Em vez de decorar endpoints fixos, o agente pergunta "quais ferramentas eu tenho?" e escolhe a certa em tempo real
- Se a sua API vira uma ferramenta usada por um agente de IA, testar bem o contrato dela importa ainda mais

---

MAPEAMENTO DA CHAMADA

Anatomia de um teste de API

- Onde bater — qual endpoint e qual método HTTP serão utilizados
- O que mandar — headers, parâmetros e corpo da requisição, quando existirem
- O que validar — status code, formato do corpo, headers relevantes e, às vezes, tempo de resposta
- Sem mapear isso antes, o teste vira tentativa e erro — e sem saber por que passou ou falhou

> *Separar a configuração da requisição das validações facilita a leitura e a manutenção do teste.*

Boas práticas de asserção

- Valide só o que importa pro teste — campos que mudam sempre (timestamp, id gerado) tendem a quebrar teste à toa
- Prefira dados de teste isolados — um teste não deveria depender do que o outro deixou pra trás
- Cadeia de chamadas (usar a resposta de uma pra montar a próxima) é poderosa, mas aumenta o acoplamento — use com cuidado
- Testes de API não substituem o E2E (end-to-end, ponta a ponta) pela tela — eles cobrem a parte que o E2E não precisa repetir

---

O que fica depois de hoje

- Interface gráfica ou código são meios diferentes para garantir que a API responde como esperado
- O raciocínio de mapear chamada, dados e validações vale para qualquer ferramenta de automação de API

Dúvidas?